REDBELLY NETWORK DAO

Community Task Board Initiative

Pilot Roster: Expanded Task Specifications

15 Tasks | Full Problem Statements | Technical Requirements
Quality Benchmarks | Failure Criteria | Budget Breakdown

Compiled by njay (@njaybby on Discord)
April 2026

This document accompanies the Community Task Board Initiative proposal. It expands all 15 pilot tasks with complete problem statements, technical requirements, scope of work, deliverable specifications, quality benchmarks, failure criteria, and infrastructure references. Budget includes 20% reviewer compensation as a separate line item.

TASK 1: SYBIL-PROOF ERC-20 (ANTI-BOT STANDARD)

Name: Sybil-Proof ERC-20 (Anti-Bot Standard)

Type: Developer Work

Reward: $40 USD equivalent in RBNT

Reviewer Compensation: $8 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Standard ERC-20 tokens have no identity requirements. Airdrop farmers create thousands of wallets to claim rewards multiple times. Traditional solutions use centralized whitelists that are expensive to maintain and create data honeypots. On a compliance-first chain like Redbelly, this problem is solvable at the protocol layer using the existing eligibility infrastructure rather than bolting on external identity solutions after the fact.

Technical Requirements

- Build an ERC-20 token contract that blocks unverified users from minting or transferring tokens
- Integrate with Redbelly's hasChainPermission(address) function from the EligibilitySDK
- Users must complete KYC through the IndividualOnboardingSDK before interacting with the token
- Contract must revert with clear, descriptive error messages when unverified users attempt restricted actions
- Configurable option allowing the contract owner to toggle whether transfers (not just minting) require verification
- Admin function to update the eligibility contract address in case Redbelly deploys updated infrastructure

Required Deliverables

1. **Smart Contract:** ERC-20 with verification gating on minting and a configurable transfer gate. Must inherit from OpenZeppelin ERC-20 and integrate cleanly with the EligibilitySDK. Deployment script targeting Redbelly Testnet (Chain ID 153).
2. **Frontend Example:** React component embedding the IndividualOnboardingSDK widget. Must display real-time verification status using the useHasChainPermission hook. UI must clearly disable token actions for unverified users with explanatory messaging directing them to the KYC flow.
3. **Test Suite:** Unit tests proving: verified users can mint successfully, unverified users are blocked from minting, transfer gating can be toggled on/off by the contract owner, the eligibility contract address can be updated by the owner but not by non-owners. Minimum 90% test coverage.
4. **Documentation:** 5 to 7 page integration guide covering contract architecture, deployment steps on Redbelly Testnet, frontend integration walkthrough, and a troubleshooting section for common errors (gas estimation failures, SDK widget rendering issues, RPC timeout handling).

Quality Benchmarks

- ✓ Deploy to Redbelly Testnet with verified source code on the block explorer
- ✓ Unverified wallet attempting to mint must revert with a KYC-specific error message
- ✓ Same wallet after completing KYC must successfully mint
- ✓ Gas cost for the verification check must not exceed 50,000 gas
- ✓ Test coverage must be 90% or higher

Failure Criteria

- Contract allows minting or transfers by unverified wallets
- Error messages are generic (e.g., 'transaction reverted') rather than KYC-specific
- Frontend example does not compile or render the SDK widget correctly
- Test suite does not cover the toggle between gated and ungated transfers
- Documentation omits deployment instructions or contains broken code snippets

Infrastructure to Use

- useHasChainPermission hook: https://docs.redbelly.network/pages/eligibility-sdk/client/hooks/useHasChainPermission/
- IndividualOnboardingSDK widget for KYC flow
- Averer's on-chain permission verification

Attachments: [None]

TASK 2: COMPLIANT ASSET TOKENIZATION (CAT) VAULT

Name: Compliant Asset Tokenization (CAT) Vault

Type: Developer Work

Reward: $60 USD equivalent in RBNT

Reviewer Compensation: $12 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Tokenized real-world assets (real estate, private equity, regulated securities) require region-specific transfer restrictions to comply with securities laws. A Manhattan apartment token can only be sold to US-accredited investors. A German bond can only be held by EU residents. Traditional solutions use off-chain databases that break composability and require manual reconciliation every time a transfer happens. On Redbelly, jurisdiction data is already stored on-chain through the Business Identifier contracts, making compliance enforceable at the smart contract layer.

Technical Requirements

- Build an ERC-4626 vault contract that restricts deposits and withdrawals based on user jurisdiction
- Integrate with the BusinessOnboardingSDK's Business Identifier contracts for institutional depositors
- Integrate with the IndividualOnboardingSDK for retail depositors
- Query business details using getBusinessDetails(address) to extract jurisdiction data
- Maintain a configurable allowlist and blocklist of jurisdictions (ISO country codes)
- Admin functions to add or remove allowed jurisdictions without redeployment

Jurisdiction Data Challenge

Business Identifier contracts store companyAddress containing location information. The contributor must choose one of the following approaches and document the trade-offs:

- **Option A:** Parse jurisdiction from the existing companyAddress string field
- **Option B:** Propose adding an explicit jurisdiction field to Business Identifier contracts

Both approaches are acceptable. The contributor must document their choice, explain the rationale, and address the limitations of the chosen approach.

Required Deliverables

1. **Smart Contract Suite:** ERC-4626 vault with jurisdiction checking, Business Identifier interface, jurisdiction helper functions, and deployment scripts for Redbelly Testnet.
2. **Jurisdiction Handling Implementation:** Complete code for extraction and validation of jurisdiction data from on-chain records, including edge case handling for malformed addresses, missing data, and jurisdictions not in the allowlist.

3. **Admin Dashboard Mockup:** Figma file OR React component showing vault configuration interface, jurisdiction management panel (add/remove jurisdictions), and transfer history with jurisdiction flags.
4. **Test Suite:** Unit tests proving: allowed jurisdictions can deposit, blocked jurisdictions are rejected, jurisdiction list updates take effect immediately, dual-path verification (Business vs. Individual SDK) routes correctly. Minimum 90% test coverage.
5. **Documentation:** 8 to 10 page guide covering the legal context for jurisdictional compliance, contract architecture, deployment walkthrough, jurisdiction configuration instructions, and compliance reporting capabilities.

Quality Benchmarks

- ✓ Deploy vault to Redbelly Testnet
- ✓ Configure with allowedJurisdictions = ["US"]
- ✓ Attempt deposit from wallet with jurisdiction "SG" (Singapore): must revert with jurisdiction error
- ✓ Deposit from "US" wallet must succeed
- ✓ Event logs clearly show jurisdiction checks for every deposit and withdrawal attempt
- ✓ Gas cost for jurisdiction check must not exceed 100,000 gas
- ✓ Test coverage must be 90% or higher

Failure Criteria

- Vault allows deposits from blocked jurisdictions
- Jurisdiction parsing fails silently instead of reverting with a clear error
- No event emissions for jurisdiction checks (audit trail missing)
- Documentation does not explain the jurisdiction data approach chosen or its trade-offs

Infrastructure to Use

- useBusinessDetails hook: https://docs.redbelly.network/pages/eligibility-sdk/client/hooks/useBusinessDetails/
- BusinessOnboardingSDK for KYB verification
- Business Identifier contracts storing company data

Attachments: [None]

TASK 3: DIVIDEND AUTOMATION CONTRACT WITH PROOF OF ELIGIBILITY

Name: Dividend Automation Contract with Proof of Eligibility

Type: Developer Work

Reward: $60 USD equivalent in RBNT

Reviewer Compensation: $12 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Tokenized REITs and bonds must verify each recipient has valid KYC at the exact moment of dividend payment, not just when they purchased the token. A holder who was compliant last quarter may have let their credentials lapse. Traditional transfer agents charge $10,000 or more per quarterly distribution for 1,000 holders. On Redbelly, the on-chain eligibility check makes this verification automatable and near-costless.

Technical Requirements

- Build an ERC-20 token with snapshot capability for recording holder balances at a point in time
- Implement a two-step dividend process:
  - Step 1 (Snapshot): Record token balances at dividend declaration date
  - Step 2 (Distribution): Pay dividends based on snapshot but check CURRENT eligibility status before each payment
- Skip payments to holders whose credentials expired since the snapshot was taken
- Allow holders to self-claim dividends (alternative to batch distribution by the issuer)
- Track unclaimed funds from expired credentials; allow the issuer to reclaim after a configurable waiting period (default 90 days)

Critical Verification Logic

At distribution time, for each recipient:

1. Calculate dividend based on snapshot balance
2. Check current eligibility status using hasChainPermission(address)
3. If eligible: transfer payment immediately
4. If expired: skip payment, emit event with reason code, and add amount to unclaimed pool
5. Track skipped amounts for later reclaim by the issuer after the waiting period expires

Required Deliverables

1. **Smart Contract:** ERC-20 with snapshot extension, dividend cycle management (declare, distribute, claim, reclaim), batch distribution function with configurable batch sizes, self-claim function, and unclaimed fund reclaim after waiting period. Deployment scripts for Redbelly Testnet.

2. **Frontend Dashboard (Optional but Recommended):** React component showing: current allocation per holder, eligibility status indicator, claim button with countdown, and distribution history.
3. **Test Suite:** Unit tests proving: snapshot accurately captures balances, dividend math is correct to the wei, eligible holders receive payment, ineligible holders are skipped with events, self-claim works, double-claim is blocked, reclaim triggers only after the waiting period. Minimum 90% test coverage.
4. **Gas Optimization Analysis:** Document showing gas costs for distributions of 20, 50, 100, and 500 holders. Include batch size recommendations for minimizing total gas cost.
5. **Documentation:** 10 to 12 page guide covering the regulatory rationale for point-in-time eligibility checks, the full process walkthrough from snapshot to distribution to reclaim, admin procedures, and gas optimization strategies.

Quality Benchmarks

✓ Deploy to Redbelly Testnet with 20 test holders
✓ Take snapshot, then expire 5 holders' credentials
✓ Execute distribution: 15 payments succeed, 5 skipped with events emitted
✓ Math check: sum of (15 payments) + (5 skipped amounts) = total dividend pool (within 1 wei tolerance)
✓ Gas per verified recipient must not exceed 80,000 gas
✓ Test coverage must be 90% or higher

Failure Criteria

- Dividend math is incorrect (total distributed plus total skipped does not equal total pool)
- Ineligible holders receive payment
- Self-claim allows double-claiming
- No events emitted for skipped payments (breaks audit trail)

Infrastructure to Use

- OpenZeppelin's ERC20Snapshot extension
- Redbelly's hasChainPermission(address) for eligibility checking
- Standard ERC-20 for payment token (e.g., USDC or testnet equivalent)

Attachments: [None]

TASK 4: CREDENTIAL EXPIRY MONITOR AND AUTO-REVOKE SYSTEM

Name: Credential Expiry Monitor and Auto-Revoke System

Type: Developer Work

Reward: $100 USD equivalent in RBNT

Reviewer Compensation: $20 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

When an investor's annual KYC lapses, securities regulations require immediate revocation of trading privileges. Manual tracking across thousands of holders is impossible at scale. Any delay between credential expiry and access revocation creates a compliance gap that regulators will scrutinize. This system must run autonomously, detect status changes, and enforce restrictions without human intervention.

Technical Requirements

Component 1: Smart Contracts

- Registry contract tracking which tokens require credential monitoring
- Freezable token extension adding freeze and unfreeze functionality to any ERC-20
- Access control allowing only the authorized keeper service to freeze accounts
- Clear event emissions for all freeze and unfreeze actions with timestamps and reason codes

Component 2: Automation Service

- Daily check of all monitored token holders
- Query hasChainPermission for each holder and compare against the previous state
- Call freezeAccount() if credential expired (status changed from true to false)
- Call unfreezeAccount() if credential renewed (status changed from false to true)

Automation Implementation Options (Choose ONE)

- **Option A: Gelato Network:** Configure Gelato Automate task running daily. Implement Web3 Function checking credentials. Document setup and gas cost management. (Use this if Gelato supports Redbelly.)
- **Option B: Custom Keeper Service:** Build Node.js service running on schedule (cron or AWS Lambda). Store previous permission states in a database or file. Call freeze/unfreeze based on state changes. Include deployment guide for AWS Lambda, DigitalOcean, or similar.
- **Option C: The Graph + Webhooks (Advanced):** Deploy subgraph indexing hasChainPermission changes. Configure webhooks triggering on state changes. (Only choose this if The Graph supports Redbelly Network.)

Required Deliverables

1. **Smart Contract Suite:** Registry contract, Freezable token extension, example token integration demonstrating both contracts working together, and deployment scripts for Redbelly Testnet.
2. **Automation Service:** Complete implementation of ONE chosen option with full source code, configuration files, state management logic, error handling, and retry logic for failed transactions.
3. **Test Suite:** Contract unit tests plus an integration test proving that 10 out of 50 accounts freeze correctly when credentials expire, then 5 out of 10 unfreeze when credentials are renewed. Frozen accounts must not be able to transfer. Minimum 90% test coverage.
4. **Monitoring Dashboard Mockup:** Figma file OR React component showing: monitored tokens list, frozen account counts, recent freeze/unfreeze events with timestamps, upcoming credential expirations (7-day and 30-day warnings), and keeper service health status.
5. **Documentation:** 15 to 18 page guide covering: regulatory context explaining why automated revocation is critical, complete system architecture diagrams, deployment guide for the chosen automation option, registry management instructions, token integration walkthrough (adding FreezableToken to existing ERC-20), monitoring and alerting setup, disaster recovery procedures, and gas cost projections.

Quality Benchmarks

- ✓ Deploy registry and test token with 50 holders (all verified) to Redbelly Testnet
- ✓ Expire credentials for 10 holders
- ✓ Run keeper service: exactly 10 accounts frozen with events emitted
- ✓ Frozen accounts cannot transfer (transaction must revert)
- ✓ Renew 5 credentials, run keeper again: 5 accounts unfrozen, 5 remain frozen
- ✓ Gas per freeze operation must not exceed 60,000 gas
- ✓ Keeper runs reliably for 7 consecutive days without manual intervention
- ✓ Test coverage must be 90% or higher

Failure Criteria

- Keeper misses a credential expiry (holder remains unfrozen after KYC lapses)
- Frozen accounts can still transfer tokens
- Keeper service crashes without automatic recovery
- No event logs for freeze/unfreeze actions (audit trail broken)

Infrastructure to Use

- Redbelly's hasChainPermission(address) function
- Standard ERC-20 transfer hooks for freeze logic
- Gelato Network OR AWS Lambda OR The Graph (chosen based on Redbelly support)

Attachments: [None]

TASK 5: "COMMUNITY SHOWCASE" WEB PAGE REDESIGN

Name: "Community Showcase" Web Page Redesign

Type: Design Work

Reward: $80 USD equivalent in RBNT

Reviewer Compensation: $16 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

The official Redbelly website (https://redbelly.network/community) lacks a dedicated, engaging space to highlight community contributions. Valuable ecosystem tools like Majnoon's TVL dashboard and Robbie's Node dashboard get lost in Discord channels. This misses a critical opportunity to validate builders, prove network activity to external visitors, and create visible momentum.

Scope of Work

- Audit the current Community page and identify structural gaps
- Map out a new page layout that seamlessly integrates into Redbelly's existing visual identity (typography, colour palette, spacing, component styles)
- Design a dynamic Showcase Grid UI component featuring:
  - Project thumbnail or screenshot
  - Builder handle and credit
  - Short description (2 to 3 sentences)
  - External link to live tool or GitHub repo
  - Optional: Technology stack badges (React, Solidity, etc.)
- Design a "Submit Your Project" call-to-action flow explaining how new builders can get featured
- Propose additional community-aligned content sections (e.g., Featured Builder Spotlight, Ecosystem Stats, Recent Contributions)

Required Deliverables

Complete Figma Project File

- Desktop layout (1920px width)
- Tablet layout (768px width)
- Mobile layout (375px width)
- All three breakpoints showing responsive behaviour

Asset Organisation

- All components properly grouped and labelled
- Layers named for developer handoff (e.g., "Showcase_Grid_Container," "Project_Card_01")
- Colour styles extracted and documented (hex codes)
- Typography styles documented (font families, weights, sizes, line heights)

- Grid system clearly marked (columns, gutters, margins)

Developer Handoff Package

- Exportable assets (icons, illustrations, mock project screenshots)
- Spacing and padding specifications
- Hover states for interactive elements
- Component variant documentation (empty state, loading state, populated state)

Quality Benchmarks

✓ Brand Consistency: Must strictly match the existing Redbelly website aesthetic. Do NOT reinvent the visual identity.

✓ Developer-Ready: Technical reviewer must be able to export all assets and implement layout without design clarification questions.

✓ Featured Projects: Must visually showcase Majnoon's TVL dashboard and Robbie's Node dashboard as core examples in the grid.

✓ Two-Click Accessibility: Users should reach featured community tools in 3 clicks or fewer from the homepage.

Infrastructure to Use

- Figma (required for deliverable format)
- Existing Redbelly Network website for brand reference: https://redbelly.network

Attachments: Referenced works by Majnoon and Robbie

TASK 6: REDBELLY INSIGHTS BRAND INTRO SEQUENCE (AUDIO + VISUAL)

Name: Redbelly Insights Brand Intro Sequence (Audio + Visual)

Type: Video and Motion Graphics

Reward: $120 USD equivalent (Split 70% RBNT / 30% USDT)

Reviewer Compensation: $24 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

The "Redbelly Insights" YouTube and podcast series (featuring interviews with institutional partners and ecosystem builders) currently lacks a high-production video intro. Without a consistent sonic and visual signature, the series fails to establish immediate brand recall and institutional authority. The first 5 to 10 seconds of video content sets the tone for credibility. Redbelly Insights needs a bumper sequence that signals TradFi-grade professionalism before the interviews begin.

Scope of Work

- Create a 5 to 10 second 3D or 2D motion graphics bumper sequence to open all Redbelly Insights episodes
- Integrate visual metaphors representing Redbelly's unique technical architecture:
  - Example 1: "DBFT finality" shown as digital locks or vault mechanisms snapping into place with precision
  - Example 2: "Receptor scanning" represented by verification grids, identity nodes being authenticated, or secure credential flows
  - Example 3: Network nodes forming a synchronized consensus pattern
- Incorporate original or properly licensed audio/sound design that conveys:
  - Institutional security (sharp, precise audio cues)
  - Technological sophistication (digital/synthetic tones)
  - Trustworthiness (avoid hype or pump aesthetics)
- Design should reference institutional finance media aesthetics (Bloomberg Technology, Fireblocks brand videos, Chainalysis reports) NOT retail crypto influencer content

Required Deliverables

Finalised Video Files

- 1080p MP4 (H.264 codec, suitable for YouTube)
- 4K MP4 (H.264 codec, future-proofed for high-resolution distribution)
- Duration: 5 to 10 seconds exactly
- Audio: Stereo mix, -14 LUFS loudness standard

Open Project Files

- After Effects project file (.aep) OR Premiere Pro (.prproj) OR Blender (.blend) if 3D
- All assets organised (fonts, audio stems, shape layers, imported media)
- Render settings documented

Asset Package

- Individual audio stems (background tone, SFX, etc.)
- Any custom 3D models or vector graphics used
- Font files if custom typography included

Quality Benchmarks

✓ Seamless Integration: Animation must loop cleanly into a transparent background OR fade to solid colour (black or dark blue) for easy overlay onto interview footage.

✓ Institutional Aesthetic: Must feel appropriate for a financial compliance webinar or central bank presentation. Avoid cartoon or anime styles, meme culture references, overly colourful DeFi aesthetics, and laser eye or rocket imagery.

✓ Technical Quality: No render artefacts, clean motion blur, smooth keyframe easing, professional colour grading.

✓ Audio Licensing: All audio must be original composition OR properly licensed. Include licence documentation. No copyright-infringing music.

Reference Examples (Aesthetic Targets)

- Bloomberg Quicktake intro sequences
- Fireblocks company explainer videos
- Chainalysis Crypto Crime Report intro
- Traditional financial institution brand videos (avoid retail crypto YouTube)

Infrastructure to Use

- After Effects, Premiere Pro, Blender, or Cinema 4D (contributor's choice)
- Audio production software (Ableton, Logic Pro, or similar)
- Proper licensing for any stock assets (Envato, Artlist, Epidemic Sound, etc.)

Attachments: [None]

TASK 7: THE "PROJECT ACACIA" DEEP-DIVE ANALYSIS

Name: The "Project Acacia" Deep-Dive Analysis

Type: Research and Analysis

Reward: $150 USD equivalent in RBNT

Reviewer Compensation: $30 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Redbelly Network's involvement in the Reserve Bank of Australia's (RBA) Project Acacia CBDC pilot is a massive institutional authority signal. However, the broader crypto and TradFi markets lack a comprehensive technical breakdown proving why Redbelly was selected and how it outperforms alternative blockchain architectures for central bank digital currency applications. This research gap prevents the ecosystem from fully leveraging the RBA partnership for credibility and business development momentum.

Scope of Work

Research Phase

- Analyse all publicly available information on Project Acacia (RBA press releases, pilot reports, technical whitepapers)
- Review Redbelly's official technical documentation to understand architectural advantages
- Compare Redbelly's DBFT consensus against alternatives tested in other CBDC pilots (Ethereum, Hyperledger, R3 Corda, etc.)

Analysis Phase

- Explain WHY Redbelly was chosen for this specific use case (transaction finality requirements, regulatory compliance integration, throughput benchmarks)
- Breakdown of technical advantages: DBFT deterministic finality vs. probabilistic finality, Receptor identity system integration for regulated participants, and atomic settlement for simultaneous payment-vs-payment in multi-party transactions
- Document real-world outcomes from the pilot (if publicly disclosed)
- Assess regulatory implications for CBDCs built on Redbelly infrastructure across at least three jurisdictions (Australia, Singapore, and one additional)

Writing Phase

- Format analysis for institutional audience (equity research report style, not promotional blog post)
- Structure with executive summary, methodology, findings, and technical appendix
- Include charts and diagrams where helpful (system architecture, consensus comparison tables, transaction flow diagrams)

Required Deliverables

1. **Comprehensive Research Report:** 15 to 20 pages (including charts and diagrams). PDF format (professional typesetting, not raw Google Docs export). Markdown source file (for future web publication). Must include executive summary, methodology, findings, comparative analysis, regulatory implications, and strategic recommendations.
2. **Proper Citations:** All claims must cite sources (RBA publications, Redbelly technical docs, academic papers on consensus mechanisms). Footnotes or endnotes for references. Complete bibliography/works cited section.
3. **Live Publication:** Published on one of: SSRN (academic repository), Mirror.xyz (crypto-native), or Substack (professional newsletter). Must be publicly accessible (not paywalled).
4. **Executive Brief:** Condensed 2 to 3 page summary suitable for distribution to institutional partners who will not read the full report.

Quality Benchmarks

✓ Professional Writing: Must read like an equity research analyst report or IMF working paper.

✓ Objective Analysis: Focus on technical facts and architectural trade-offs. Avoid promotional language without evidence-backed justification.

✓ Technical Depth: A central bank technology officer or blockchain architect should learn NEW information from reading this, not just marketing talking points.

✓ Citation Accuracy: Technical reviewer must be able to verify all factual claims via provided sources.

Specific Questions the Report Must Answer

1. What were the RBA's technical requirements for Project Acacia?
2. How does Redbelly's DBFT consensus provide finality advantages over probabilistic consensus?
3. What role did the Receptor identity system play in pilot compliance requirements?
4. How does Redbelly's atomic settlement compare to traditional RTGS (Real-Time Gross Settlement) systems?
5. What are the implications for future CBDC deployments on Redbelly infrastructure?

Infrastructure to Use

- SSRN, Mirror.xyz, or Substack for publication
- Academic citation management (Zotero, Mendeley) recommended
- Diagramming tools (Figma, draw.io) for technical illustrations

Attachments: [None]

TASK 8: "EXISTING BRIDGE INTEGRATION GUIDE"

Name: "Existing Bridge Integration Guide"

Type: Developer Documentation

Reward: $80 USD equivalent in RBNT

Reviewer Compensation: $16 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Limited cross-chain interoperability makes Redbelly Network feel isolated. Users trying to bring external liquidity or assets (USDC, WETH, etc.) from major chains like Ethereum or Polygon face friction due to a lack of clear bridge integration documentation. This creates an onboarding barrier for DeFi users and limits TVL growth.

Scope of Work

1. Research Phase

- Identify which existing cross-chain bridges currently support Redbelly Network (if any)
- Test compatibility of major bridge protocols: LayerZero, Axelar, Wormhole, Connext, and Multichain (if still operational)
- Determine if Redbelly Network can be added as a custom chain to any of these protocols

2. Implementation Phase

- Set up a testnet bridge integration (Ethereum Sepolia to Redbelly Testnet OR Polygon Mumbai to Redbelly Testnet)
- Document the exact architecture required: RPC endpoints, contract deployments (if custom bridge contracts are needed), gas token requirements, and message verification mechanisms
- Execute a successful test transaction moving a token cross-chain

3. Documentation Phase

- Write a step-by-step tutorial allowing a mid-level developer to replicate the integration
- Include a troubleshooting section for common errors (RPC timeouts, gas estimation failures, bridge fee calculation, stuck transactions)

Required Deliverables

1. **Technical Guide (Markdown):** Introduction explaining bridge architecture basics, prerequisites (wallets, testnet funds, RPC access), step-by-step integration walkthrough, code snippets for key operations (approve, bridge, claim), troubleshooting section, and gas cost analysis for different transaction sizes.
2. **GitHub Repository:** Working test script OR minimal frontend demonstrating a successful bridge transaction. README with setup instructions. Environment variables template (.env.example). All dependencies documented. Test transaction hash(es) proving successful cross-chain transfer on testnet.

3. **Published Article:** Guide published on Dev.to OR Medium. Title optimised for search. Include screenshots or screen recordings where helpful.

Quality Benchmarks

- ✓ Functional Accuracy: A mid-level developer with basic Web3 experience must be able to successfully bridge a test token from Ethereum Sepolia to Redbelly Testnet in under 1 hour by following the guide.
- ✓ Code Quality: All code snippets must be fully commented, properly formatted, and free of deprecation warnings.
- ✓ Testnet Proof: Must include verifiable testnet transaction hashes showing successful bridge execution.
- ✓ Troubleshooting Coverage: Guide must address at least 5 common error scenarios with exact solutions.

Failure Criteria

- Guide contains broken RPC endpoints or deprecated contract addresses
- Code examples throw errors when executed
- No testnet transaction proof provided
- Tutorial skips critical steps (e.g., does not explain how to acquire testnet RBNT for gas)

Infrastructure to Use

- Redbelly Testnet RPC: https://rpc-testnet.redbelly.network
- Ethereum Sepolia OR Polygon Mumbai testnet
- LayerZero, Axelar, or alternative bridge SDKs
- Hardhat or Foundry for testing scripts

Attachments: [None]

TASK 9: THE "ANTI-FRICTION" DEVELOPER TROUBLESHOOTING WIKI

Name: The "Anti-Friction" Developer Troubleshooting Wiki

Type: Developer Documentation

Reward: $60 USD equivalent in RBNT

Reviewer Compensation: $12 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Redbelly's current developer documentation is heavily theoretical and research-focused. Developers encountering standard technical roadblocks (RPC connection timeouts, gas estimation errors, SDK integration failures, contract deployment issues) are forced to rely on trial-and-error or wait hours for Discord support responses. This creates unnecessary friction and slows ecosystem development.

Scope of Work

Research Phase

- Review Redbelly Developer Discord and Telegram channels for recurring technical questions
- Identify the 15 to 20 most common error patterns and roadblocks developers encounter
- Categorise issues by type: Network/RPC connection problems, smart contract deployment failures, EligibilitySDK integration errors, gas estimation and transaction failures, wallet connection issues, and testnet faucet problems

Documentation Phase

For each identified issue, create a structured troubleshooting entry:

- **Symptom:** Exact error message or observable behaviour
- **Root Cause:** Why this error occurs
- **Solution:** Step-by-step fix with exact commands or code changes
- **Prevention:** How to avoid this issue in future projects

Required Deliverables

1. **Structured Troubleshooting Wiki (Markdown):** Organised by category. Minimum 15 to 20 documented issues. Each entry formatted consistently with Symptom, Root Cause, Solution, and Prevention sections. Quick-reference index allowing developers to search by error message or keyword.
2. **Live Published Link:** Published on GitHub Wiki, Dev.to, or Medium. Must be publicly searchable with good SEO title and metadata. Include table of contents for easy navigation.
3. **Community Validation:** Share draft in Redbelly Discord developer channel. Incorporate feedback from at least 3 active developers.

Example Issue Coverage (Not Exhaustive)

- RPC endpoint returning 429 (rate limit) errors
- "Nonce too low" transaction failures
- MetaMask not detecting Redbelly Network
- EligibilitySDK widget not rendering
- Hardhat deployment script failing with "insufficient funds"
- Contract verification on block explorer failing
- Gas price estimation returning null
- Testnet faucet not distributing RBNT
- Cross-origin issues with SDK iframe
- Transaction pending indefinitely

Quality Benchmarks

✓ Actionability: Solutions must be concrete ("Run this command") not vague ("Check your configuration").

✓ Scannability: Formatting must enable quick problem identification (clear headers, code blocks, bullet points).

✓ Accuracy: Technical reviewer tests at least 5 solutions to verify they actually fix the stated problems.

✓ Coverage: Must address issues accounting for more than 80% of Discord support questions based on channel analysis.

Infrastructure to Use

- Redbelly Developer Discord and Telegram for research
- Redbelly Technical Documentation for context
- GitHub Wiki, Dev.to, or Medium for publication
- Redbelly Testnet for testing solutions

Attachments: [None]

TASK 10: "REDBELLY NETWORK DASHBOARD AND TOKEN UTILITY HUB"

Name: "Redbelly Network Dashboard and Token Utility Hub"

Type: Web Development and Research

Reward: $120 USD equivalent in RBNT

Reviewer Compensation: $24 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Redbelly Network suffers from an invisibility problem on major DeFi aggregators. Partnership activities and tokenized asset values are not tracked on DeFiLlama or RWA.xyz, making the network appear inactive to external researchers and investors. The network's actual on-chain activity is healthy but unreported, creating a perception gap that hurts business development and community morale.

Scope of Work

Interactive Network Dashboard

Build a public-facing web application displaying real-time Redbelly Network metrics.

On-Chain Data (Live via RPC or Indexer)

- Total Value Locked (TVL) across all deployed protocols
- Total transactions processed (24h, 7d, 30d, all-time)
- Active addresses (daily and monthly)
- Total gas fees paid in RBNT
- Smart contracts deployed
- Average block time and finality metrics

Off-Chain Data (Manually Curated or API-Sourced)

- Number of verified businesses (via BusinessOnboardingSDK)
- Number of verified individuals (via IndividualOnboardingSDK)
- List of tokenized RWA projects on Redbelly (if publicly disclosed)
- Partnership announcements (RBA Project Acacia, enterprise integrations)

UI/UX Requirements

- Responsive design (desktop, tablet, mobile)
- Real-time or near-real-time data updates (WebSocket or polling)
- Charts and graphs for time-series data (TVL over time, transaction volume trends)
- Clean, institutional aesthetic matching Redbelly branding
- Fast load times (under 3 seconds initial page load)

Required Deliverables

1. **Live Hosted Dashboard:** Deployed to Vercel, Netlify, or similar (must have a live URL). Open-source GitHub repository with complete source code. README with local setup instructions. Environment variables template (.env.example). Documented data sources.
2. **Technical Documentation:** Architecture overview (how data flows from blockchain to dashboard). API documentation if custom endpoints are created. Deployment guide. Maintenance procedures (how to add new protocols to TVL calculation).

Quality Benchmarks

✓ On-chain data must be functionally accurate (reviewer verifies TVL matches manual calculation)
✓ UI must be responsive on mobile devices (tested on iPhone and Android)
✓ Page load time under 3 seconds on standard broadband connection
✓ Charts must display properly across all major browsers (Chrome, Firefox, Safari, Edge)
✓ No console errors in browser developer tools
✓ Follows Redbelly's visual brand guidelines

Technical Stack Recommendations

- Frontend: React + Next.js (for SSR and SEO benefits)
- Data Fetching: ethers.js or viem for RPC calls
- Charts: Recharts, Chart.js, or D3.js
- Styling: Tailwind CSS or styled-components
- Hosting: Vercel (recommended for Next.js) or Netlify

Infrastructure to Use

- Redbelly Mainnet RPC: https://rpc.redbelly.network
- Redbelly Block Explorer for contract address verification

Attachments: [None]

TASK 11: "RBNT TOKEN UTILITY AND ECOSYSTEM VISIBILITY REPORT"

Name: "RBNT Token Utility and Ecosystem Visibility Report"

Type: Research and Analysis + Ecosystem Building

Reward: $100 USD equivalent in RBNT

Reviewer Compensation: $20 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Community members and external observers lack clarity on how institutional adoption actually drives RBNT token demand beyond basic gas fee utility. Additionally, Redbelly is absent from major RWA tracking platforms (RWA.xyz, DeFiLlama), limiting discoverability and making the network appear less established than it is. This is a research and strategic deliverable that complements the Dashboard (Task 10) with the analytical layer investors and partners need.

Scope of Work

Part 1: RBNT Tokenomics Explainer

- Gas Fees: How RBNT is used for transaction costs
- Staking Mechanics: If and how RBNT can be staked for network security or rewards
- Supply Dynamics: Total supply, emission schedule, burn mechanisms (if any)
- Demand Drivers: How institutional adoption increases RBNT demand (e.g., "Every verified business on-chain requires RBNT for gas. More businesses equals more RBNT demand.")
- Network Effects: Relationship between TVL growth and token value
- Governance: If RBNT holders have voting rights in DAO decisions
- Must align strictly with Redbelly's official whitepaper and tokenomics documentation
- Avoid speculative price predictions or promotional language

Part 2: DeFiLlama and RWA.xyz Submission Kit

- Research exact integration requirements for both platforms
- DeFiLlama: TVL adapter code, API endpoints, contract addresses
- RWA.xyz: Asset registry format, verification process
- Draft submission-ready integration code: TVL calculation scripts, API endpoints for data fetching, contract ABI and address mappings, and testing documentation proving data accuracy
- Write a submission guide explaining how to maintain listings (data updates, refresh schedules)

Required Deliverables

1. **RBNT Token Utility Report:** 10 to 15 pages including charts and diagrams. Professionally formatted PDF. Must cite Redbelly's official whitepaper for all claims. Competitive comparison table showing RBNT utility against tokens of other RWA-focused Layer 1 networks.

2. **Explainer Article:** Under 500 words, titled "How Network Adoption Drives RBNT Value," formatted for community distribution on social media and Discord.
3. **DeFiLlama Submission Kit:** Complete TVL adapter code (JavaScript or TypeScript). API endpoint documentation. Contract address registry (JSON format). Testing guide proving adapter accuracy. Submission instructions document.
4. **RWA.xyz Submission Kit:** Asset registry data in the required format. Verification documentation. Submission walkthrough with exact contact points and application timeline.

Quality Benchmarks

✓ All tokenomics claims must cite official Redbelly documentation
✓ No speculative price predictions or investment advice
✓ TVL adapter code executes without errors
✓ Calculated TVL matches manual verification within 5% margin
✓ Submission plans must be actionable within 30 days of delivery
✓ Suitable for presentation to institutional investors (professional tone and formatting)

Infrastructure to Use

- DeFiLlama adapter documentation: https://docs.llama.fi
- RWA.xyz integration docs (if available)
- Redbelly official whitepaper and tokenomics documentation

Attachments: [None]

TASK 12: THE COMMUNITY MANDATE: X ACCOUNT MANAGER

Name: The Community Mandate: X Account Manager

Type: Community Management (Monthly Recurring)

Reward: $150 USD equivalent in RBNT per month (this budget covers Month 1 only; subsequent months billed separately)

Reviewer Compensation: N/A (recurring role; oversight handled through monthly reporting and DAO review rather than per-task review)

Assign To User: [Leave blank]

Reputation Points: N/A (ongoing role)

Description

Problem

The Redbelly DAO's X (Twitter) account needs consistent, high-quality management that balances institutional credibility with community warmth. The account must amplify technical work produced by task board contributors, engage with ecosystem partners, and attract developers and builders, all while maintaining a voice that resonates with both TradFi professionals and crypto-native builders. Poor social media management leads to low engagement, missed partnership opportunities, and wasted content production efforts.

Scope of Work

Daily Responsibilities

- **Content Creation and Scheduling:** Minimum 1 to 2 tweets per day (or threads where appropriate). Weekly thread highlighting completed task board deliverables. Translate technical outputs (smart contracts, research reports, SDK guides) into digestible, engaging content. Share ecosystem updates, partnership announcements, and builder spotlights.
- **Community Engagement:** Respond to mentions and DMs within 24 hours. Proactively engage with ecosystem builders, RWA/DeFi protocols that could integrate with Redbelly, institutional blockchain projects, and Web3 developer communities. Amplify community-created content (tutorials, tools, analyses).
- **Reputation Management:** Monitor for FUD and address with facts. Correct misinformation about Redbelly Network capabilities. Professional crisis communication if needed.

Content Guidelines

- **Institutional Voice (70% of content):** Focus on technical achievements, partnership milestones, and compliance advantages. Use professional language appropriate for central banks and enterprises. Cite verifiable sources. Avoid hype language.
- **Community Warmth (30% of content):** Celebrate builder contributions and community wins. Share ecosystem culture when appropriate (but tastefully). Humanise the DAO with behind-the-scenes content. Engage in friendly interaction with partner projects.

Required Deliverables

1. **Ongoing Account Management:** Daily content creation and posting (1 to 2 tweets or threads per day minimum). Active community engagement (responses, retweets, quote tweets). DM monitoring and professional responses to partnership inquiries.

2. **Monthly Analytics Report:** Submitted by the 5th of the following month. Includes follower growth, impression metrics, engagement rate, link clicks, and community sentiment analysis. Highlights top-performing content and explains what worked. Recommendations for next month's content strategy.
3. **Content Calendar (Optional but Recommended):** Weekly content plan showing scheduled tweets. Helps DAO members preview upcoming announcements. Allows collaboration on major announcements.

Engagement KPIs (Minimum Targets)

- ✓ Follower growth: +10% per quarter
- ✓ Average engagement rate: above 2% (likes + retweets + replies / impressions)
- ✓ Response time: under 24 hours to mentions and DMs during business hours
- ✓ Content consistency: Minimum 25 tweets per month (approximately 1 per weekday)
- ✓ Partnership engagement: Minimum 3 meaningful interactions with potential partners per month

Content Quality Standards

- ✓ Zero typos or grammatical errors in tweets
- ✓ All factual claims must be verifiable (link to source or block explorer)
- ✓ Professional tone maintained even when addressing criticism
- ✓ No engagement with obvious scams or malicious accounts

Blue Checkmark Requirement

X Premium subscription ($8/month) must be maintained for the verification badge. This is funded directly by the DAO treasury and is outside of the $150/month retainer.

Revocability Clause

This role is revocable by DAO vote if any of the following occur: Account goes dormant (more than 3 consecutive days without activity). Engagement metrics fall more than 30% below agreed KPIs for 2 consecutive months. Tone deviates significantly from brand guidelines. Failure to submit monthly analytics reports. If revoked, the contributor receives pro-rated payment for completed weeks only.

Onboarding Requirements

- Contributor must provide X account handle for verification
- Must demonstrate previous social media management experience (portfolio of past accounts managed)
- Sign off on brand voice guidelines document
- Complete handoff call with Alison or designated DAO member

Infrastructure to Use

- X (Twitter) platform
- Social media scheduling tools (Buffer, Hootsuite, TweetDeck) optional
- Analytics tools (Twitter Analytics, optional third-party tools)

Attachments: [None]

TASK 13: "ZERO-TO-HERO" DEVELOPER ONBOARDING KIT

Name: "Zero-to-Hero" Developer Onboarding Kit

Type: Developer Documentation and Video Production

Reward: $150 USD equivalent (Split 70% RBNT / 30% USDT)

Reviewer Compensation: $30 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

Multiple community survey responses flagged that Redbelly's current documentation lacks linear, step-by-step guidance for complete beginners. Existing docs assume prior blockchain development knowledge and jump straight into advanced concepts. New builders need a highly structured curriculum that takes them from zero setup to their first deployed contract in a single, cohesive learning path, similar to how Solana has "Cookbook" or Ethereum has "CryptoZombies."

Scope of Work

Part 1: Written Interactive Tutorial Series

Design a linear curriculum with 5 progressive modules:

- **Module 1: Environment Setup:** Installing Node.js and package manager. Setting up Hardhat or Foundry. Configuring MetaMask for Redbelly Testnet (Chain ID 153). Acquiring testnet RBNT from faucet. Connecting to Redbelly Testnet RPC.
- **Module 2: Your First Smart Contract:** Writing a basic contract in Solidity. Understanding contract structure. Compiling with Hardhat. Deploying to Redbelly Testnet. Verifying contract on block explorer. Interacting with deployed contract via Hardhat console.
- **Module 3: Reading and Writing State:** Building a simple storage contract. Understanding state variables vs. memory. Writing and reading data. Gas costs for state changes. Event emission and logging.
- **Module 4: Access Control and Modifiers:** Creating an ownable contract. Implementing role-based access control. Using OpenZeppelin libraries. Function modifiers (onlyOwner, etc.). Security best practices.
- **Module 5: Real-World Integration:** Building a simple ERC-20 token. Integrating with Redbelly's EligibilitySDK (KYC-gated token example). Deploying multi-contract systems. Frontend integration basics. Gas optimisation techniques.

Each module must include: learning objectives, step-by-step instructions, fully commented code examples, expected outputs, common errors and troubleshooting, and a quiz or checkpoint to verify understanding.

Part 2: GitHub Repository

- 5 working code examples (one per module), each in its own folder with a complete Hardhat project structure, README, deployment script, test suite, and .env.example template
- Root README with full curriculum overview

Part 3: Video Walkthrough Series

- 3 to 5 HD screen-capture videos (one per module or grouped logically)
- Each video: 10 to 20 minutes, 1080p minimum resolution, clear audio, legible on-screen code (minimum 16pt font)
- Show actual coding process (not just final code), narrate thought process, display terminal outputs in real-time

Required Deliverables

1. **Written Tutorial Series:** 5 comprehensive modules in Markdown format. Published on GitHub (in repository root as docs/). Optionally also published on Dev.to or Medium for discoverability.
2. **GitHub Repository:** 5 working Hardhat projects (one per module). All code thoroughly commented. Each project runnable without modifications (assuming user has RPC access). Root README with installation and curriculum overview.
3. **Video Series:** 3 to 5 videos uploaded to YouTube. Organised in a playlist titled "Redbelly Network: Zero-to-Hero Developer Course." Video descriptions include GitHub repo link and written tutorial links. Chapter markers in video timeline.

Quality Benchmarks

✓ A developer with zero prior Redbelly experience but basic JavaScript knowledge must be able to deploy a working smart contract to testnet in under 2 hours using this kit

✓ Technical reviewer follows entire curriculum start-to-finish and verifies: all code examples compile without errors, all deployment scripts work on current testnet, all expected outputs match actual outputs

✓ Audio is clear with no background noise or distortion

✓ On-screen code is legible at 1080p

✓ No deprecated libraries or outdated syntax

Failure Criteria

- Code examples do not compile or deploy
- Videos have poor audio quality or illegible code
- Tutorial skips critical steps (e.g., how to get testnet RBNT)
- Assumes knowledge not taught in earlier modules (breaks linear progression)

Infrastructure to Use

- Redbelly Testnet (Chain ID 153)
- Redbelly Testnet RPC: https://rpc-testnet.redbelly.network
- Hardhat (preferred) or Foundry, Solidity ^0.8.20 or later, OpenZeppelin Contracts v5.x
- Video Recording: OBS Studio, Loom, or ScreenFlow. Editing: DaVinci Resolve, Premiere Pro, or Final Cut Pro.

Attachments: [None]

TASK 14: "ELIGIBILITYSDK" INTEGRATION GUIDE

Name: "EligibilitySDK" Integration Guide

Type: Developer Documentation

Reward: $100 USD equivalent in RBNT

Reviewer Compensation: $20 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

The EligibilitySDK is critical infrastructure for any application that needs to verify user eligibility on Redbelly, but it lacks a dedicated frontend integration guide. Developers are left guessing on backend verification flows, widget embedding patterns, and error handling. This slows down adoption of Redbelly's core compliance tooling and means every developer independently rediscovers the same integration pitfalls.

Scope of Work

- Write a comprehensive step-by-step guide covering the full EligibilitySDK implementation path
- Frontend widget embedding (IndividualOnboardingSDK and BusinessOnboardingSDK)
- Backend verification using hasChainPermission and getBusinessDetails
- End-to-end testing on Redbelly Testnet
- React and Next.js (App Router) code examples
- Integration patterns for combining the EligibilitySDK with both Onboarding SDKs
- Complete error resolution section mapping every documented SDK error code to its cause and fix

Required Deliverables

1. **Step-by-Step Integration Guide:** Covers the full path from frontend widget embedding through backend verification to production deployment. Includes architecture diagrams showing data flow between frontend, backend, and on-chain contracts.
2. **React Code Examples:** Working examples demonstrating EligibilitySDK widget integration with the useHasChainPermission and useBusinessDetails hooks. Must include component code, state management, loading states, and error handling.
3. **Next.js (App Router) Code Examples:** Server-side verification patterns using the App Router. Must demonstrate both static and dynamic rendering approaches.
4. **Integration Patterns Document:** How to combine the EligibilitySDK with the IndividualOnboardingSDK (for retail users) and the BusinessOnboardingSDK (for institutional users). Decision tree helping developers choose the right SDK combination for their use case.
5. **Error Resolution Section:** Every documented SDK error code mapped to its cause, example scenario, and step-by-step fix. Must be formatted for quick scanning (error code as heading, solution as body).

6. **Published Guide:** All materials formatted for publication on Dev.to or Medium. Must be publicly accessible and searchable.

Quality Benchmarks

- ✓ A developer should be able to integrate the EligibilitySDK widget into an existing dApp within 4 hours following the guide
- ✓ The error resolution section must cover every error code documented in the official SDK reference
- ✓ All code examples must compile and run without modification on current versions of React and Next.js
- ✓ Technical reviewer confirms that the integration patterns document correctly routes developers to the appropriate SDK for their use case

Infrastructure to Use

- EligibilitySDK documentation: https://docs.redbelly.network/pages/eligibility-sdk/
- useHasChainPermission hook documentation
- useBusinessDetails hook documentation
- IndividualOnboardingSDK and BusinessOnboardingSDK widgets
- Redbelly Testnet for end-to-end testing

Attachments: [None]

TASK 15: DAO WEBSITE NAVIGATION OVERHAUL AND INFORMATION ARCHITECTURE

Name: DAO Website Navigation Overhaul and Information Architecture

Type: Ecosystem Building (Design and Strategy)

Reward: $150 USD equivalent in RBNT

Reviewer Compensation: $30 USD equivalent in RBNT (20% of task reward, paid separately from budget)

Assign To User: [Leave blank]

Reputation Points: 10

Description

Problem

The current website architecture is fragmented. Finding bridge support, developer documentation, or DAO governance details requires deep digging through multiple pages with inconsistent navigation patterns. This friction frustrates developers, confuses potential institutional partners conducting due diligence, and wastes the time of community members who should be building rather than searching.

Scope of Work

- Redesign the entire site information architecture to establish clear, intuitive pathways for three primary audiences:
  - **Developers:** Documentation, SDKs, code examples, testnet resources, API references
  - **DAO Members:** Governance, proposals, task board, voting, treasury transparency
  - **Institutional Users:** Case studies, compliance information, partnership details, Project Acacia credentials
- Create a navigation flowchart mapping every page on the current site to its new location
- Develop a cross-linking strategy explaining how related content across sections will be connected
- Build user journey maps for each primary audience showing the expected click path from homepage to target content

Required Deliverables

1. **Complete Figma Mockup:** Redesigned site architecture showing all primary and secondary navigation paths. Must include desktop and mobile navigation patterns.
2. **Navigation Flowchart:** Visual map of every page on the current site and its proposed new location. Must clearly show which pages are being merged, split, renamed, or removed.
3. **Cross-Linking Strategy Document:** Explains how related content across sections will be connected. For example, a developer documentation page linking to the relevant SDK guide and the DAO task that produced it.
4. **User Journey Maps:** One map per primary audience (Developers, DAO Members, Institutional Users) showing the expected click path from homepage to their most common target content.
5. **Before-and-After Comparison:** Annotated comparison showing specific improvements to findability for the 10 most visited content types on the current site.

Quality Benchmarks

- ✓ A user should be able to find bridge support, DAO governance information, or developer documentation in fewer than 2 clicks from the homepage
- ✓ A usability reviewer must confirm that the proposed architecture eliminates the fragmentation problems identified in the current site
- ✓ The navigation flowchart must account for every existing page (no orphaned content)
- ✓ Cross-linking strategy must identify at least 20 connection points between sections
- ✓ Mobile navigation must be tested and documented separately from desktop

Failure Criteria

- Proposed architecture introduces new dead ends or orphaned pages
- Navigation flowchart is incomplete (missing pages from the current site)
- No mobile navigation consideration
- Cross-linking strategy is generic rather than identifying specific page-to-page connections

Infrastructure to Use

- Figma for mockups and flowcharts
- Existing Redbelly Network website for audit baseline: https://redbelly.network
- Google Analytics or similar (if access is provided) for identifying the 10 most visited content types

Attachments: [None]

BUDGET AND CAPACITY BREAKDOWN (MONTH 1)

This budget covers the first month of the pilot programme. Task 12 (X Account Manager) is billed at $150 per month; only Month 1 is included in this budget. Subsequent months are billed separately. The X Account Manager role does not carry a reviewer stipend because oversight is handled through monthly analytics reporting and DAO review rather than per-task review.

#	Task Title	Contributor	Reviewer	Split	Combined	Category
1	Sybil-Proof ERC-20	$40	$8	RBNT	**$48**	D
2	CAT Vault	$60	$12	RBNT	**$72**	D
3	Dividend Automation	$60	$12	RBNT	**$72**	D
4	Credential Expiry Monitor	$100	$20	RBNT	**$120**	D
5	Community Showcase Redesign	$80	$16	RBNT	**$96**	B+E
6	Redbelly Insights Intro	$120	$24	70/30	**$144**	B
7	Project Acacia Analysis	$150	$30	RBNT	**$180**	C
8	Bridge Integration Guide	$80	$16	RBNT	**$96**	A
9	Troubleshooting Wiki	$60	$12	RBNT	**$72**	A
10	Network Dashboard	$120	$24	RBNT	**$144**	D+C
11	RBNT Token Utility Report	$100	$20	RBNT	**$120**	C+E
12	X Account Manager (Month 1)	$150	N/A	RBNT	**$150**	B
13	Zero-to-Hero Onboarding Kit	$150	$30	70/30	**$180**	A
14	EligibilitySDK Guide	$100	$20	RBNT	**$120**	A
15	Website Navigation Overhaul	$150	$30	RBNT	**$180**	E
	TOTALS	**$1,520**	**$274**		**$1,794**	

Budget Summary (Month 1 Only)

Total Contributor Rewards: $1,520 USD equivalent

Total Reviewer Compensation (20%): $274 USD equivalent

Total Committed (Month 1): $1,794 USD equivalent

All amounts are expressed in USD equivalent. RBNT payments are converted to a fixed RBNT number at proposal approval using the 7-day moving average. Video tasks (Tasks 6 and 13) include a 30% USDT component to cover software, stock assets, and hosting costs. The USDT component is ring-fenced and cannot be used for any other task type.

Reviewer Compensation Policy

Reviewer compensation is set at 20% of the contributor reward for each task. This amount is paid separately from the contributor's reward and is drawn directly from the quarterly budget. The reviewer receives compensation upon completion of their review duties (initial evaluation, feedback, and final approval or rejection). Reviewer payments are scaled by task complexity to ensure proportional

compensation. The X Account Manager role (Task 12) is exempt from reviewer compensation because oversight is conducted through monthly reporting and DAO review rather than individual task review.

Quarterly Projection

If all 15 tasks are completed in Month 1 and the X Account Manager continues for the full quarter at $150/month, the projected quarterly spend is:

Month 1 (this budget): $1,794

Month 2 (X Account Manager only): $150

Month 3 (X Account Manager only): $150

Projected Quarterly Total: $2,094 USD equivalent

Additional tasks may be added in Months 2 and 3 based on demand and available budget.

DELIVERABLE PUBLICATION WORKFLOW

The following framework establishes where each category of deliverable will be published and how the DAO will track completion.

1. Engineering and Smart Contracts

All code submissions go to a public GitHub repository where the review committee will audit them. Once approved, companion step-by-step guides are published on Dev.to or Medium so builders can find them without digging through raw codebases.

2. Research and Analysis

Heavy institutional reports (such as the Project Acacia analysis) are submitted to SSRN (Social Science Research Network) to secure institutional credibility and Google Scholar indexing. Authors cross-post to Mirror.xyz or Substack to capture Web3 ecosystem readership.

3. Design and UI/UX

UI mockups and dashboard designs are published as open-source files on Figma Community. This allows anyone in the ecosystem to inspect, duplicate, and use the approved templates.

4. Video and Motion Graphics

All video tutorials and motion sequences are hosted permanently on YouTube for long-term SEO and easy embedding. The X Account Manager pulls the highest-leverage clips for distribution on X (Twitter) and LinkedIn.

ADMINISTRATIVE RECORD-KEEPING

To maintain transparency for the DAO without exposing contributor identities, record-keeping is split into two layers.

The Private Backend

A locked Google Sheet or private GitHub Projects board, accessible only to admins. This holds contributor emails, Discord handles, wallet addresses, and raw revision histories.

The Public Transparency Ledger

A read-only, filtered view generated from the backend for the DAO community. The public ledger displays four columns only: Task ID, Current Status, Exact RBNT Payout, and Link to Final Deliverable. No personal identities or draft materials are exposed. This gives the community an audit trail of treasury ROI without turning the review process into a surveillance mechanism.